

# SafeAIMerge: A Tool for Integrating DAST and LLM-Generated Security Feedback into GitHub Actions Workflows

Arpit Thool  
Virginia Tech  
Blacksburg, USA  
arpitthool@vt.edu

Justin Smith  
Lafayette College  
Easton, USA  
smithjus@lafayette.edu

Chris Brown  
Virginia Tech  
Blacksburg, USA  
dcbrown@vt.edu

## Abstract

Continuous integration and delivery (CI/CD) emphasizes automated workflows and rapid deployment, yet the integration of security practices into these workflows remains a persistent challenge. For instance, traditional Dynamic Application Security Testing (DAST) tools are effective for identifying web application vulnerabilities, yet these complex systems are difficult to integrate in CI/CD workflows and often produce lengthy and complex reports that are difficult to understand. To address this gap, we present *SafeAIMerge*, a tool that integrates DAST within GitHub pull requests by leveraging LLMs to generate actionable and developer-friendly security alert summaries. We conducted a two-phase study to motivate and evaluate *SafeAIMerge*. First, we distributed a formative survey to capture developers' initial perceptions of our tool ( $n = 46$ ). We observed favorable perceptions and received valuable feedback to enhance our tool in real-world settings. We refined our tool, then conducted a user study ( $n = 12$ ) to assess *SafeAIMerge* in debugging tasks. We found *SafeAIMerge* reduces perceived developer workload and enhances vulnerability remediation compared to baseline DAST workflows. Based on our findings, we provide insights to improve the integration of DAST practices into modern development pipelines. *SafeAIMerge* is accessible on GitHub ([github.com/arpitthool/SafeAIMerge](https://github.com/arpitthool/SafeAIMerge)).

## CCS Concepts

• Security and privacy → Web application security; Software security engineering; Social aspects of security and privacy; Usability in security and privacy; Vulnerability scanners.

## Keywords

Dynamic Application Security Testing (DAST), Large Language Models (LLM), Report Summarization, GitHub-Actions

## ACM Reference Format:

Arpit Thool, Justin Smith, and Chris Brown. 2026. SafeAIMerge: A Tool for Integrating DAST and LLM-Generated Security Feedback into GitHub Actions Workflows. In *Proceedings of the 2026 ACM Secure Development Conference (SecDev '26)*, July 5–6, 2026, Montreal, QC, Canada. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3805773.3806003>



This work is licensed under a Creative Commons Attribution 4.0 International License. SecDev '26, Montreal, QC, Canada

© 2026 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-2602-6/2026/07  
<https://doi.org/10.1145/3805773.3806003>

## 1 Introduction

Modern software development increasingly relies on continuous integration and delivery (CI/CD) practices that streamline development through automated building, testing, and deployment [1, 22, 42]. Although these practices have accelerated software development, embedding security measures within these fast-paced workflows continues to pose significant difficulties [18, 21]. Security-related tasks that add overhead, slow down feedback loops, or demand expert knowledge tend to be sidelined in favor of shipping features quickly [5, 9]. A notable example is Dynamic Application Security Testing (DAST) tools—tools such as ZAP<sup>1</sup> and Burp Suite<sup>2</sup> that mimic hackers' techniques to uncover vulnerabilities in live web applications [16, 45]. But their adoption faces several obstacles. Reports produced by these tools are often verbose, highly technical, and challenging for developers without a security background to understand [10, 53]. Furthermore, fitting them into developer-oriented pipelines such as CI/CD remains largely unresolved [39].

The above challenges are further amplified in contemporary development environments, where keeping documentation lean and maintaining swift feedback loops are considered essential [33, 37]. Large language models (LLMs), machine learning models trained on large amounts of data, offer promising capabilities to make technical reports more accessible and usable through processing, summarizing, and generating natural language [24, 38, 65]. For instance, prior work suggests LLMs can provide security summaries tailored for developers [53] and actionable security guidance from static testing in development workflows [2]. Integrating these results within CI/CD pipelines—such as those built on GitHub Actions—can make DAST more accessible to development teams. As far as we are aware, existing literature has not yet explored the combination of DAST tools with LLM-driven report summarization and fix recommendations within CI/CD workflows. This drives our work toward enhancing both usability and actionability of DAST tools.

To this end, we introduce *SafeAIMerge*, a lightweight security tool integrated into CI/CD pipelines that couples DAST with LLM-driven summaries and fix-suggestions to deliver developer-focused security feedback in GitHub Pull Request (PR). Built on top of GitHub Actions *SafeAIMerge* leverages the ZAP and runs automatically each time a PR is opened or modified. The system, when triggered, executes DAST scan and generates a report consisting of alerts. These alerts are then combined with the PR code changes and provided to an LLM, which generates concise, PR-aware summaries and actionable remediation guidance, helping compensate for DAST's limited source-code context. The aggregated results

<sup>1</sup><https://www.zaproxy.org/>

<sup>2</sup><https://portswigger.net/burp/pro>

are delivered directly within the PR through a structured comment and an accompanying HTML report artifact, enabling rapid, context-aware security feedback that aligns with core CI/CD principles such as fast feedback, continuous integration, and shared understanding [4, 13, 20, 44].

We evaluated *SafeAImerge* via two empirical phases. During the initial stage, we distributed a web-based questionnaire to 46 software professionals to get their early impressions regarding adoption potential, clarity and usability. Findings revealed a clear preference among developers for LLM-produced summaries vs. traditional DAST output, along with strong support for embedding AI-driven security insights directly in PRs. This exploratory stage functioned as a formative assessment, to guide iterative improvements to *SafeAImerge*. Based on responses, we enhanced the reporting and presentation of output to better integrate *SafeAImerge* in developer workflows. The second stage involved a controlled task-based user study in which 12 developers performed predefined tasks using both the standard ZAP workflow and the *SafeAImerge* workflow. We assessed workload through the NASA Task Load Index (NASA-TLX), recorded task outcomes, and collected their subjective preferences. Our findings demonstrate that *SafeAImerge* lowers the cognitive burden on developers, enables them to identify and remediate security vulnerabilities more quickly and accurately, and delivers a better workflow experience for handling security issues within CI/CD environments. Based on our results, we discuss implications to motivate human-centered designs to promote system security and DAST integration into modern development workflows. The contributions of this work include:

- *SafeAImerge*, a tool for integrating DAST into CI/CD by providing actionable LLM-generated summaries embedded in GitHub Actions workflows.
- Preliminary formative and empirical results demonstrating *SafeAImerge*'s ability to reduce developer workload and enhance security vulnerability remediation in codebases.

## 2 Background

**Dynamic Application Security Testing (DAST):** DAST is a black-box technique that probes web applications and APIs while they are actively running [36]. Unlike Static Application Security Testing (SAST), which inspects the source code itself [17], DAST operates externally, mimicking the behavior of a real-world attacker, identifying range of security flaws—e.g. [Cross-Site Scripting \(XSS\)](#), [SQL Injection](#) etc.—by sending requests to live applications and assessing their response [43].

Zed Attack Proxy (ZAP) is a freely available, open-source security scanner designed to work across multiple platforms [46] and is maintained by Checkmarx.<sup>3</sup> It offers both automated scanning functionality and a hands-on interface that supports manual exploration and in-depth analysis. Once a scan concludes, DAST tools produce comprehensive vulnerability reports organized according to severity. *SafeAImerge* builds upon ZAP to integrate DAST in CI/CD workflows.

**GitHub Actions:** GitHub Actions is the default automated workflow and CI/CD platform for GitHub repositories, allowing developers to automate building and testing activities. Studies show GitHub Actions is one of the most popular CI/CD platforms in practice [19]. Recent work characterizes GitHub Actions usage and impacts in open-source development [27, 64]. For instance, Decan et al. show most projects incorporate multiple workflows, yet security is one of the least prevalent categories [15]. GitHub Actions automates tasks through workflows triggered by various activities,<sup>4</sup> such as PR created on repositories.

## 3 Related Work

Prior research consistently shows that the effectiveness of security tools in modern development workflows depends not only on their detection capabilities, but on how findings are communicated and integrated into developers' everyday practices. Studies of security testing in Agile and CI/CD environments report that developers often struggle to interpret verbose, technically dense reports and to translate findings into concrete remediation actions, particularly during code review [13, 39]. Braz and Bacchelli [10] found that security issues raised during modern code review frequently lack actionable context, increasing cognitive load and delaying fixes. Similarly, Smith et al. [48] demonstrated that usability barriers in static analysis tools prevent developers from understanding and resolving reported vulnerabilities, even when issues are accurately detected. This highlights that poor workflow integration and limited actionability remain central obstacles to adopting security practices in fast-paced CI/CD pipelines.

An increasing number of studies have investigated how LLMs can augment software security analysis, particularly in SAST. Prior studies propose hybrid architectures in which LLMs reason over SAST outputs to improve vulnerability detection, reduce false positives, or provide explanations for reported issues [26, 29, 61]. Large-scale comparisons show that LLM-based approaches often achieve high recall but at the cost of increased false positives, reinforcing the need for careful integration rather than wholesale replacement of existing tools [67]. Moving beyond benchmark-driven evaluations, Steenhoek et al. [50] evaluated an IDE-integrated LLM-based vulnerability detection and repair tool with professional developers. They found that false positives and non-applicable fixes remain major barriers due to missing contextual information and limited project-specific customization, underscoring persistent challenges around trust and workflow integration. Overall, this literature primarily emphasizes detection quality and tool accuracy, with limited attention to how security feedback affects developer workload, remediation efficiency, or CI/CD-based code review practices.

*SafeAImerge* builds on these insights by shifting the focus from detection-centric improvements to developer-centered security remediation in CI/CD workflows. As far as we are aware, existing research has not yet brought together DAST with PR-aware, LLM-assisted security feedback and evaluates its impact on developer workload, remediation effectiveness, and workflow preference. By grounding its design and evaluation in empirical studies of developer experience, we complement prior SAST- and LLM-based

<sup>3</sup><https://checkmarx.com/>

<sup>4</sup><https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows>

research by demonstrating how human-centric, workflow-aligned security feedback can reduce usability barriers and improve the practical effectiveness of security testing in modern CI/CD environments.

## 4 SafeAImerge

### 4.1 Design

The design of *SafeAImerge* was guided by recurring challenges reported in the literature and our prior studies on security tool adoption in CI/CD environments: (a) DAST outputs are often lengthy, difficult to interpret, and create significant cognitive overhead for developers who lack security expertise [10, 53, 62]; (b) integrating security practices into CI/CD pipelines is perceived as disruptive, demanding additional documentation or slow feedback cycles [21, 39]. Based on these challenges, we derived four core design principles for *SafeAImerge*: (i) automation; (ii) developer-centric integration; (iii) human-centric reporting; and (iv) lightweight adoption. These principles informed the key features, given below.

**Automated Security Scanning in PRs:** To eliminate the need for manual security checks, *SafeAImerge* automatically initiates a ZAP-based DAST scan whenever a PR is opened or updated with new commits. Prior studies [7, 17, 55] emphasize the importance of iterative and incremental vulnerability assessment in Agile development, showing that automation lowers adoption barriers and supports more consistent security practices. Modern development methodologies, such as Agile, prioritize quick feedback loops and incremental delivery, principles that are put into practice via CI/CD pipelines automating the build, test, and release stages [1, 22, 42]. DevSecOps extends this by weaving security checks directly into the pipeline, shifting vulnerability detection to earlier phases of development instead of deferring it to final-stage reviews [40, 52].

**LLM-Assisted Summarization of Security Alerts:** In our prior research, we showed that developers often perceive conventional DAST output as overly lengthy and hard to interpret, expressing a clear preference for brief, LLM-generated overviews instead [53]. To lessen the mental effort required of developers, *SafeAImerge* passes every DAST alert to an LLM, which in turn generates an explanation of the vulnerability, a recommended remediation fix, and a high-level summary. The LLM prompts in *SafeAImerge* are augmented with relevant PR details, such as code changes, whether it is a new or resolved vulnerability or a pre-existing one, present already in the main branch. Providing the code changes allows the LLM to produce more targeted and practical guidance by relating detected vulnerabilities directly to the modifications under review. This strategy puts into practice the growing demand for security reporting that is both developer-focused and sensitive to project context [48, 53], ultimately producing summaries that are more accessible to those without specialized security expertise and more straightforward to act upon as part of everyday development processes.

**Effortless Setup and Lightweight Adoption:** Prior work shows that developers are often reluctant to adopt security tools that force context switching or impose substantial workflow disruption [39, 55, 56]. To reduce this friction, *SafeAImerge* is embedded directly into GitHub and its CI/CD pipeline, surfacing security feedback within the PR interface where code review and discussion

already occur. The tool posts structured comments on the PR that display the discovered issues in categories, separating newly introduced, resolved, and existing alerts via dropdowns; results are further grouped by severity and paired with concise summaries to support rapid triage and remediation. Alongside the PR comments, *SafeAImerge* also creates a downloadable HTML report, providing issue summaries, remediation guidance, and a browsable interface while preserving detailed vulnerability information for deeper analysis and auditability. To keep the setup lightweight, *SafeAImerge* integrates natively with GitHub Actions<sup>5</sup> and is distributed as a self-contained `.security` directory that can be added to an existing repository with minimal configuration. Deployment is further simplified through Docker<sup>6</sup>, which encapsulates dependencies and configuration, ensuring consistent execution across heterogeneous environments, reducing environment-related failures, and lowering ongoing maintenance overhead. Together, PR-native feedback, CI/CD integration, and containerized deployment lower the barrier to adoption for fast-moving teams while supporting organizational needs for traceable security records.

### 4.2 Implementation

*SafeAImerge* is architected as a set of modules within a CI/CD pipeline hooked directly into GitHub's PRs, allowing it to be adapted to a variety of repositories without much overhead. Figure 1 illustrates the sequence of operations, which we explain in detail below.

**Integration with GitHub-Actions:** Central to the system is a GitHub-Actions workflow file (`zap.yaml`) which defines the orchestration logic for running DAST scans using ZAP in response to PR updates. The workflow is automatically triggered when: (i) a PR targeting the main branch is created or updated; (ii) scheduled Cron<sup>7</sup> job runs; or (iii) manually run through the GitHub-Actions interface (see Figure 1: 1 and 2). The job provisions a self-hosted runner, starts the target web-application/API (e.g., OWASP Juice Shop) inside a Docker container and also launches another container running ZAP (3). This approach guarantees reproducible and isolated runs. After the environment is ready, the workflow installs dependencies, executes the main Python scanning script, and uploads the generated security report as a GitHub artifact.

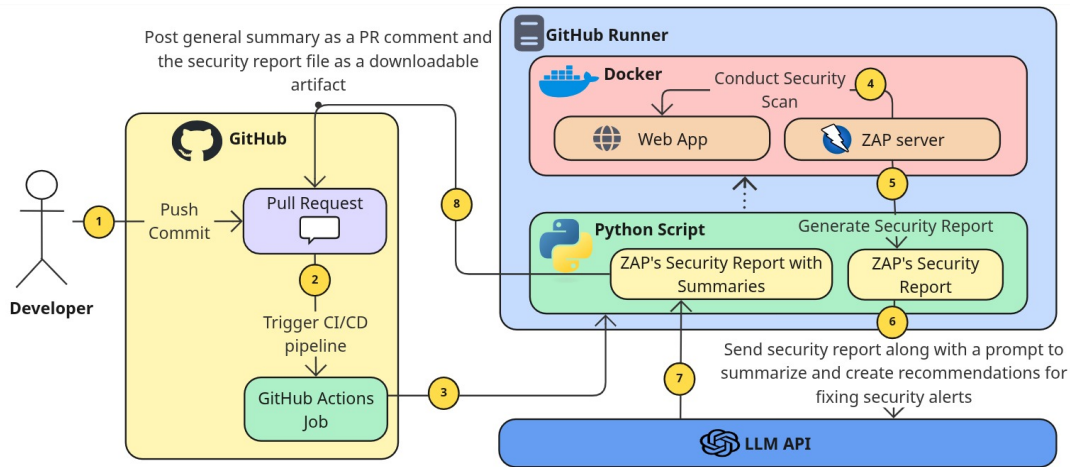
#### **Dual-Branch Scanning and Comparative Analysis:**

*SafeAImerge* executes a pair of coordinated security scans for each PR in the GitHub Runner (4). One scan is run against the application version associated with the main branch, while the second targets the version built from the PR branch. This paired scanning strategy allows the system to focus on security changes brought in by the PR, rather than reporting vulnerabilities in isolation. Once both scans complete, their results are systematically compared to categorize findings as (i) newly introduced by the PR, (ii) mitigated or resolved by the PR, and (iii) pre-existing vulnerabilities that persist unchanged. By framing findings in terms of these categories, *SafeAImerge* makes the security implications of code changes explicit, supporting developer judgment during code review and better aligning security feedback with PR-centric development workflows.

<sup>5</sup><https://github.com/features/actions>

<sup>6</sup><https://www.docker.com/>

<sup>7</sup><https://en.wikipedia.org/wiki/Cron>



**Figure 1: Overview of the system architecture and data-flow pipeline for *SafeAIMerge*. The numbered arrows (1–8) trace the sequential execution steps triggered by PR modifications.**

**Configurable Scan Policies:** To support different project security postures, the tool relies on a YAML configuration file that allows developers to specify preferences such as: (i) which ZAP scans to conduct (passive, active, spider or AJAX spider); (ii) risk levels to summarize (Medium, High); (iii) risk levels to ignore (e.g., Low); (iv) thresholds that fail the pipeline (e.g., blocking merge on High level alerts); and (v) maximum number of alerts to include in a single report. This design decouples scanning policies from the workflow logic, making *SafeAIMerge* lightweight to adopt and adaptable across different teams.

**Automated Scanning and Alert Collection:** The main scanning logic resides in a scanning module, which interfaces with the ZAP. The script: (i) launches the selected scans; (ii) monitors their progress until completion; and (iii) retrieves all alerts from ZAP. All communication occurs via a shared Docker network, guaranteeing the scanner can effectively explore the running application.

**LLM-Based Summarization:** Detected alerts are processed by an alert processing component, which applies a configurable filtering and summarization pipeline (5). First, alerts are filtered according to the YAML configuration (ignored levels are skipped, and pipeline-blocking levels are flagged). Second, for alerts at configured severity levels, the raw JSON is sent to an LLM (GPT 5.2 in our prototype) using tailored system prompts (6). The LLM generates short summaries, fix recommendations, and highlights of key risks. A consolidated report is then created (7), which includes: (i) per-alert summaries for prioritized vulnerabilities; (ii) a statistical breakdown of all alerts by severity; and (iii) a high-level summary generated by the LLM, emphasizing urgent issues and next steps. The full report is written to `security_report.txt` and archived as an artifact for further review.

**Feedback in GitHub PRs:** Finally, the scanning module invokes a GitHub API client to post the high-level summary directly as a comment in the PR (8). The comment includes inline Markdown-formatted content showing the summarized alerts and a link to download the complete report artifact. This integration ensures that developers receive actionable security insights within the same

interface used for code review, reducing context switching and aligning with CI/CD principles.

**Extensibility and Pipeline Control:** To support common DevSecOps adoption patterns, *SafeAIMerge* was designed to be highly extensible. The LLM prompting logic is decoupled from the implementation and stored in external text files, enabling teams to tailor the wording, granularity, and remediation guidance without modifying the underlying system. In addition, configurable pipeline gating policies allow merges to be blocked when critical vulnerabilities are detected, accommodating teams with more stringent security and compliance constraints.

## 5 Phase I: Formative Study

To assess initial perceptions and motivate the approach for our tool, we conducted an online survey with 46 industry practitioners. The survey was approved by our Institutional Review Board (IRB) and distributed via our professional network.

### 5.1 Methodology

**Survey Design:** The formative survey collected both qualitative and quantitative feedback. Participants were first provided with a high-level description of the tool, a video walkthrough<sup>8</sup> demonstrating its workflow and configuration options, and a link to the GitHub repository for optional exploration. Likert-scale items assessed perceived user-friendliness, CI/CD integration ease, usefulness of GitHub PR comments, adoption likelihood, and overall satisfaction. Additional questions examined whether LLM summaries improved the comprehension of security findings, likelihood of recommending the tool, and perceived ease of integration into existing workflows. The survey also included optional open-ended questions inviting participants to comment on unclear aspects of the interface, potential barriers to adoption, particularly valuable features, suggested improvements, and any additional feedback. The full set of survey questions is publicly available online [57]. This

<sup>8</sup>*SafeAIMerge* demo video

mixed-method approach enabled a comprehensive characterization of practitioner perceptions.

**Participants:** Our survey was distributed to developers in the researchers’ network and on social media. Survey respondents spanned multiple professional roles, including DevOps engineers, software engineers, technical leads and security analysts. Participants were employed at companies such as Microsoft, Amazon, Adobe, IBM, etc. Reported technical professional years of experience averaged to 4.8 (0 to 20), with 14 participants reporting more than five and 32 indicating five or fewer.

**Data Collection and Analysis:** We used the Wilcoxon signed-rank test to analyze five-point Likert scale responses for participants’ initial perceptions of user friendliness, ease of integration, usefulness, likelihood of adoption, and overall satisfaction. To analyze open-ended responses, we conducted thematic analysis through qualitative coding to derive themes outlining initial perceptions and feedback for *SafeAIMerge*.

## 5.2 Results

**Table 1: Median, mean, Wilcoxon signed-rank test results, and effect sizes ( $r$ ) for survey responses (N=46).**

| Measure             | Mean | Median | $r$ -value | $p$ -value |
|---------------------|------|--------|------------|------------|
| User-friendliness   | 4.78 | 5      | 0.55       | < 0.001    |
| Integration ease    | 4.74 | 5      | 0.47       | < 0.001    |
| PR usefulness       | 4.78 | 5      | 0.46       | < 0.001    |
| Adoption likelihood | 4.35 | 4      | 0.37       | < 0.001    |
| Satisfaction        | 4.85 | 5      | 0.43       | < 0.001    |

**Initial Perceptions:** Table 1 reports summary statistics and inferential outcomes for the Likert-scale measures: user-friendliness, ease of integration, usefulness of PR-feedback, likelihood of adoption, and overall satisfaction. Responses were consistently positive across all dimensions, with median values ranging from 4 to 5 and mean scores between 4.35 and 4.85. Wilcoxon signed-rank tests indicated that all measures were significantly higher than the neutral midpoint of 3 ( $p < 0.001$  for each item), with effect sizes in the medium-to-large range ( $r = 0.37$ – $0.55$ ). All respondents (100%) reported that they would recommend the tool to colleagues. With respect to integration, 38 participants stated that the tool “fits in seamlessly” with CI/CD pipelines, while 8 reported that “some adjustments” would be required. The usefulness of embedding security feedback directly in GitHub PRs was also strongly supported, with 36 respondents selecting “strongly agree” and the remaining 10 selecting “agree”.

Qualitative analysis of the open-ended responses revealed several recurring themes. Participants frequently highlighted the *clarity of the LLM-generated summaries*, noting that these summaries reduced the cognitive effort required to interpret raw security alerts. Others underscored the value of *workflow compatibility*, emphasizing that GitHub PR integration minimized friction within existing development practices. Suggestions for improvement were focused on the *reporting format*, including requests for richer visualizations or dashboard-style summaries (see Section 5.3). Taken together,

these findings suggest that practitioners view *SafeAIMerge* as a lightweight and user-friendly enhancement to CI/CD workflows. The combination of high satisfaction scores, unanimous willingness to recommend the tool, and strong support for PR-based reporting highlights its practical adoption potential.

## 5.3 SafeAIMergeRefinement

Although the initial survey findings suggested that *SafeAIMerge* was positively received and fit well within existing development workflows, participants also offered concrete recommendations for enhancement. In particular, suggestions focused on improving the visual representation of generated security feedback to better integrate output into practical development workflows. Based on this feedback, we made two main modifications to our system:

**Interactive HTML Report:** *SafeAIMerge* conducts the DAST scan and provides an LLM-generated summary, attaching the generated artifacts as a plain text file (“`.txt`”). However, participants desired to receive more interactive reports in an HTML format. In response, we transformed the unstructured plain-text security report into a structured HTML report with interactive capabilities. This redesigned report categorizes results into three explicit groups: (i) vulnerabilities newly introduced by the PR, (ii) vulnerabilities resolved by the PR, and (iii) vulnerabilities that already existed on the main branch. Each category is presented as an expandable drop-down, enabling developers to reveal detailed information only when required. This approach supports rapid, high-level evaluation of security impact while preserving access to complete alert details for in-depth analysis.

**Pull Request Comment:** In addition to the text-based file report, *SafeAIMerge* automatically posts the generated summary as a review comment<sup>9</sup> on the open GitHub PR that triggered the CI/CD build. Participants also desired to have a more structured GitHub PR comment to better parse and understand the reported DAST vulnerabilities. We applied a similar redesign to the GitHub PR comment produced by *SafeAIMerge*. Rather than emitting a single text block, the updated comments now contain three expandable sections aligned with new, resolved, and pre-existing alerts. Each section provides a brief summary of its category, allowing reviewers to quickly assess the security implications of a change without wading through extensive output. This format was intentionally designed to better support common code-review workflows, which favor concise, high-signal information that can be expanded as needed. Guided by this feedback, we iteratively refined *SafeAIMerge* to enhance the tool while maintaining compatibility with developers’ established workflows.

## 6 Phase II: Summative User Study

Although the online survey offered preliminary evidence regarding perceived usability and likelihood of adoption, it did not observe how developers engage with *SafeAIMerge* to conduct security tasks in practice. Hence, we conducted a controlled user study that compared *SafeAIMerge* with a baseline ZAP-based workflow, incorporating the refinements informed by the formative survey feedback.

<sup>9</sup><https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/reviewing-changes-in-pull-requests/commenting-on-a-pull-request>

This study was designed to investigate RQs (listed below) each deliberately formulated to address the principal obstacles surfaced by earlier work and to examine factors that are pivotal to the real-world integration of security tools into CI/CD pipelines—cognitive load experienced by developers [32], how effectively vulnerabilities are resolved [58], and the overall throughput of the development workflow [66].

**RQ1** How does *SafeAImerge* affect perceived developer workload during vulnerability remediation?

**RQ2** How does *SafeAImerge* impact developers' ability to understand and resolve vulnerabilities?

**RQ3** How does the *SafeAImerge* workflow compare to traditional DAST workflows when addressing security vulnerabilities in CI/CD pipelines?

## 6.1 Methodology

To investigate the impact of *SafeAImerge* on resolving security bugs in web applications, we conducted a controlled user study [28]. This effort was also approved by our IRB.

**Study Design:** The controlled user study employed a within-subjects design with two conditions:

- (1) **Baseline ZAP:** participants interacted with a standard ZAP setup integrated into a GitHub Actions CI/CD pipeline and examined the resulting ZAP security report to identify and remediate reported vulnerabilities.
- (2) ***SafeAImerge*:** participants used the *SafeAImerge* integration within a GitHub Actions CI/CD pipeline and reviewed the generated security feedback, which included LLM-generated summaries and suggested remediation guidance embedded directly in the PR.

Each participant completed both workflows during a single study session lasting approximately 45–60 minutes. At the start of the session, the researcher outlined the study objectives and procedures. Participants were informed that all responses would be kept confidential and anonymized. Participants then received a brief primer on DAST, covering the purpose of DAST scans and their integration into GitHub Actions-based CI/CD pipelines. Finally, participants completed a short demographic questionnaire collecting background information such as organizational context, professional role, and prior CI/CD experience. All study sessions were recorded for analysis by the research team.

**Baseline Condition.** In the baseline condition, the researcher first introduced the participant to a GitHub repository<sup>10</sup> containing a lightweight TypeScript and HTML5-based web application. Together with the researcher, participants integrated standard ZAP scans into the repository's GitHub Actions CI/CD workflow and then initiated a pre-configured vulnerable PR, which triggered the ZAP scan. Once the workflow execution completed, participants reviewed the resulting ZAP security report, identified the vulnerabilities, and attempted to remediate as many issues as possible within a fixed task window of 15 minutes. Immediately after completing the baseline task—or upon reaching the time limit—participants examined the outcome of their changes (if any code had been pushed

to the PR) and then completed a NASA-TLX workload assessment covering the standard six workload dimensions.

***SafeAImerge Condition.*** In the *SafeAImerge* condition, participants proceeded to a second GitHub repository,<sup>11</sup> which hosted another simple TypeScript and HTML5 web application. Participants and the researcher integrated *SafeAImerge* into the GitHub Actions workflow and created a prepared PR to trigger the automated scans. Participants then reviewed the resulting *SafeAImerge* security report. Unlike the baseline setup, *SafeAImerge* presented summarized security findings, developer-centered remediation guidance, and an enhanced reporting experience directly within the PR context. As in the baseline condition, participants then attempted to resolve as many vulnerabilities as possible within a 15-minute time limit. After completing the task—or upon reaching the time limit—participants reviewed the outcome of their changes (if applicable) and completed the NASA-TLX workload questionnaire for a second time. To reduce potential learning and ordering effects, the study employed a counterbalanced design: six of the twelve participants completed the baseline workflow first, while the remaining six began with the *SafeAImerge* condition.

**Repository and Vulnerability Selection.** To support a fair and tightly controlled comparison of the two workflows, we configured both GitHub repositories to yield security reports with the same number, type, and severity distribution of vulnerabilities. Both repositories were similar, consisting of open-source web-based versions of popular classic video games: Pac-Man and Maze Escape. The repositories were also counterbalanced across participants. Each project was based on TypeScript and HTML5, with comparable codebase sizes and structures. As a result, any differences observed in perceived workload, task completion time, or the number of vulnerabilities remediated can be attributed to how security feedback was formatted and embedded into the workflow, rather than to any variation in the underlying scan results.

Across both conditions, the generated security reports included a mix of informational-, low-, and medium-level severity findings. Specifically, each report contained two medium-severity alert types—*Missing Anti-Clickjacking Header* and *Content Security Policy (CSP) Header Not Set*—with four instances of each. In addition, four low-severity issues were reported, including *X-Content-Type-Options Header Missing*, *Server Version Information Leakage via HTTP Headers*, *Permissions Policy Header Not Set*, and *Insufficient Site Isolation Against Spectre Vulnerability*. A small number of informational alerts related to storable or cacheable content were also present, however these alerts do not report detected vulnerabilities in security scans and were not observed in our study. In both the baseline ZAP and *SafeAImerge* conditions, the prepared PRs invoked the same ZAP scan configuration and produced reports containing an identical set of alerts. These vulnerabilities were chosen because resolving them requires participants to interpret the reported security issue, identify the appropriate configuration or middleware responsible for the behavior, and implement a specific corrective change. As such, they reflect realistic and commonly

<sup>10</sup>Baseline ZAP GitHub repository

<sup>11</sup>*SafeAImerge* GitHub repository

encountered remediation activities in web application security engineering. The user study questions are provided as supplemental material.

**Participants:** Participants were recruited through the authors' professional network. 12 individuals joined the study, spanning diverse technical positions and company settings and on average had 4.25 years of industry experience (0 to 15). 11/12 participants had prior CI/CD experience, but none had previous exposure to security-oriented workflows, enabling us to observe how developers lacking dedicated security knowledge interact with and act upon security-related feedback. Notably, this participant composition reflects industry contexts, where CI/CD adoption is widespread yet developers frequently have little hands-on experience with structured security processes [3].

**Data Collection and Analysis:** To address RQ1 concerning perceived workload, we employed a modified version of the NASA-TLX questionnaire which is widely used for subjective workload evaluation [14]. Each participant rated their experience along six workload dimensions on five-point Likert scales: (i) **Mental Demand**, capturing the degree of cognitive effort required (Very Low → Very High); (ii) **Physical Demand**, reflecting physical exertion during the task (Very Low → Very High); (iii) **Temporal Demand**, indicating how rushed or time-pressured the task felt (Very Low → Very High); (iv) **Perceived Performance**, representing participants' self-assessed success in completing the task (Highly Unsuccessful → Highly Successful); (v) **Effort**, measuring how hard participants had to work to reach their performance level (Very Low → Very High); and (vi) **Frustration**, capturing feelings of insecurity, discouragement, irritation, stress, or annoyance experienced during the task (Very Low → Very High). The NASA-TLX questionnaire was administered after each task condition to capture workload perceptions specific to each workflow. Lower scores on mental demand, temporal demand, effort, and frustration corresponded to lower perceived workload, whereas higher perceived performance scores indicated greater task success.

For each condition, we computed a raw NASA-TLX workload score by averaging the six workload dimensions after converting Likert-scale responses to a 0–100 scale and reversing the *perceived performance* dimension so that lower values consistently reflected lower workload. We then calculated condition-level descriptive statistics, including means, medians, and standard deviations, across all participants. Because the study followed a within-subjects design and the data did not satisfy normality assumptions, we used the Wilcoxon signed-rank test for paired comparisons of NASA-TLX scores across study settings. Alongside statistical significance, we report effect sizes using the rank-biserial correlation ( $r$ ), computed as  $r = Z/\sqrt{N}$ , where  $Z$  denotes the standardized Wilcoxon statistic and  $N$  represents the number of paired observations.

To answer RQ2, we recorded the number of vulnerabilities successfully remediated and measured task completion time, defined as the time required to resolve at least one vulnerability. Participants who did not remediate any vulnerabilities within the allotted time were assigned the maximum task duration. We computed the mean and the task completion time range to observe participants' efficiency across the study settings. We also compared remediation

outcomes across workflows to assess differences in task success, correlating with participants' understanding and ability to resolve vulnerabilities.

To address RQ3, participants completed a post-study questionnaire after experiencing both workflows, designed to elicit direct comparisons between the baseline ZAP and *SafeAImerge* conditions based on five dimensions: (i) **Overall ease of use**, assessing general workflow usability; (ii) **Effectiveness in supporting understanding of reported vulnerabilities and fix suggestions**, evaluating if LLM-powered, PR-aware feedback improved understanding and actionability; (iii) **Overall workflow preference**, capturing participants' holistic judgments; (iv) **Clarity and actionability of the generated security reports**, isolating the impact of report presentation and guidance quality, based on prior work investigating LLM-summarized security alerts [53]; and (v) **Perceived time required to interpret vulnerabilities**, examining if differences in feedback presentation influenced participants' perceived time pressure and efficiency, complementing objective completion-time data. For each item, participants chose one of the four options: (a) baseline ZAP; (b) *SafeAImerge*; (c) both equally; or (d) neither. We analyzed these measures using condition-level descriptive statistics (means, medians, and ranges).

## 6.2 Results

Our findings provide initial summative insights on the effectiveness of *SafeAImerge* for vulnerability remediation in codebases. Participant quotes from the study are provided with a P- prefix.

**Developer Workload:** We analyzed participants' NASA-TLX responses to compare perceived workload between the baseline ZAP workflow and the *SafeAImerge* workflow. Table 2 presents the results for each individual NASA-TLX dimension and the overall workload outcomes. Participants reported significantly lower temporal, mental, frustration, effort, and performance-related workload while using *SafeAImerge* compared to the baseline. Physical demand also decreased, but not significantly. Across every dimension showing significant difference, the effect size ( $r$ ) was large, indicating the reduction in workload was not only statistically reliable but also practically meaningful.

Participants experienced significantly lower overall workload with *SafeAImerge*. The median raw NASA-TLX score dropped from 58.3 (baseline) to 12.5 (*SafeAImerge*). This difference was statistically significant ( $W = 1.0$ ,  $p < 0.001$ ) with a large effect size ( $r = 0.85$ ). One participant emphasized that the step-by-step remediation guidance lowered effort and frustration: “one thing I like even more is how it does actual steps... it gives specific step-by-step structures on how to do solve them” (P2). In contrast, participants often described the baseline ZAP workflow as cognitively overwhelming, requiring sustained effort to interpret dense reports and identify next steps. As one participant explained, “I’m kind of confused. I don’t know what to do after looking at the report... there’s no actionable steps to actually edit the code” (P6). This sense of confusion and uncertainty aligns with the significantly higher mental demand, effort, and frustration scores reported in the baseline condition. Taken together, these findings indicate that delivering summarized, PR-contextualized security feedback directly within developers’

**Table 2: NASA-TLX workload dimensions: baseline ZAP vs. SafeAImerge.**

| Dimension           | Tool Mean | Baseline Mean | Median Diff | Wilcoxon (W) | Effect Size (r) | p-value |
|---------------------|-----------|---------------|-------------|--------------|-----------------|---------|
| Mental Demand       | 20.83     | 72.92         | -50.00      | 0.0          | 0.77            | 0.003   |
| Physical Demand     | 12.50     | 25.00         | -12.50      | 2.5          | 0.39            | 0.084   |
| Temporal Demand     | 16.67     | 39.58         | -25.00      | 0.0          | 0.69            | 0.008   |
| Performance (inv.)* | 14.58     | 77.08         | -62.50      | 0.0          | 0.79            | 0.003   |
| Effort              | 16.67     | 66.67         | -50.00      | 0.0          | 0.75            | 0.005   |
| Frustration         | 10.42     | 54.17         | -43.75      | 0.0          | 0.71            | 0.007   |
| <b>Overall</b>      | 15.28     | 55.90         | -45.83      | 1.0          | 0.85            | < 0.001 |

\* Performance scores were inverted, so that lower values indicate lower performance (more desirable).

workflows can significantly decrease the emotional and mental effort associated with security bug fixes.

**Remediation Effectiveness and Efficiency:** Successful vulnerability resolution and task completion time was analyzed for each workflow (Table 3). With standard-ZAP, every participant exhausted the entire 15-minutes (mean: 15, median: 15). While with *SafeAImerge*, participants finished considerably faster, with completion times ranging from 2 to 15 minutes (mean: 5.9, median: 5). Since all baseline times hit the maximum limit, a paired statistical comparison was uninformative. The descriptive results clearly indicate a significant and consistent reduction in time-to-action with *SafeAImerge*. Under *SafeAImerge*, 11/12 participants successfully fixed at least one vulnerability, compared to only 1/12 under standard ZAP condition. *Missing Anti-Alickjacking Header*, a medium-severity issue was the most frequently remediated. Notably, in both conditions, this alert was listed first in the security reports, as findings were sorted by severity. Two participants also addressed other low-level issues, including *Content Security Policy (CSP) Header Not Set* and *X-Content-Type-Options Header Missing*.

Observationally, with *SafeAImerge*, participants were frequently able to rely directly on the report summaries and suggested fixes presented in the PR. They moved rapidly from reviewing the report to applying a fix, with little trial-and-error. In the standard ZAP condition, however, participants devoted a considerable time understanding the raw scan results, consulting external documentation, and determining how specific code changes corresponded to the reported issues. Collectively, these findings show that *SafeAImerge* improves both the speed and the success rate with which developers convert security scan results into actionable remediation.

**Workflow Preference:** We analyzed questionnaire responses collected after participants completed both experimental conditions. For every comparison item (ease of use, understanding, preference, actionable reports, and perceived time), responses were unanimous. All (12/12) participants identified *SafeAImerge* workflow as easier to use, better in explaining of identified issues and corresponding fix suggestions, and as their overall preferred workflow. Participants also consistently reported that *SafeAImerge* provided more actionable security reports which took lesser time to comprehend compared to the baseline ZAP workflow, for instance, one participant said that, “[*SafeAImerge*] was way too better, like it directly gave me certain solutions” (P5). In contrast, participants described the baseline ZAP workflow as difficult to navigate and less supportive for decision making. As one participant explained, “this is

really confusing if we compare to the previous report... I’m a bit stuck here” (P5). Such feedback highlights how the lack of actionable guidance and dense reporting in the baseline condition negatively shaped participants’ perceptions. Collectively, these comparative findings offer compelling support that developers preferred the *SafeAImerge* augmented workflow to the conventional ZAP-only approach when handling security alerts within CI/CD workflows. When considered alongside task performance and workload, the preference data further suggest that *SafeAImerge* does more than just reduce the effort developers perceive—it is also regarded as a more supportive and practical mechanism for integrating DAST into their development workflows

## 7 Discussion

Across both phases, we found developers found DAST integration with *SafeAImerge* to be effective. The initial survey found developers perceived our tool to be user-friendly, easy to integrate, and useful, providing valuable feedback to improve reporting format. In the user study, we found *SafeAImerge* reduced perceived workload and improved remediation outcomes compared to the baseline ZAP workflow. Despite identical underlying DAST findings across conditions in our user study, participants reported lower mental demand, effort, and frustration, and were substantially more likely to remediate vulnerabilities—and to do so more quickly—when using *SafeAImerge*. These gains can be attributed to how findings were communicated rather than what was detected.

Traditional DAST tools generate comprehensive reports that often impose high cognitive and temporal overhead [53]. In addition, prior work suggests dynamic analysis and testing approaches face challenges integrating into CI/CD workflows [41]. These findings align with challenges observed in a prior real-world case study focused on DAST adoption in CI/CD, where practitioners reported DAST integration was constrained by limited developer bandwidth to parse and act on security reports [56]. *SafeAImerge* addresses this gap by embedding DAST findings directly into PRs and augmenting them with LLM-generated summaries and remediation guidance. Based on our results, we provide insights for future tools to provide developer-oriented feedback through vulnerability communication and workflow integration.

### 7.1 Vulnerability Communication

**LLM Summarization:** Our results demonstrate that concise summaries of DAST alerts can improve developers’ ability to engage

**Table 3: ZAP vs. *SafeAIMerge* Vulnerability remediation outcomes and task completion time.**

| Participant | Baseline               |                   | <i>SafeAIMerge</i>     |                   |
|-------------|------------------------|-------------------|------------------------|-------------------|
|             | No. of Alerts Resolved | Time Taken (min.) | No. of Alerts Resolved | Time Taken (min.) |
| P1          | 0                      | N/A               | 1 (Medium: 1)          | 3                 |
| P2          | 0                      | N/A               | 1 (Medium: 1)          | 2                 |
| P3          | 0                      | N/A               | 0                      | N/A               |
| P4          | 0                      | N/A               | 1 (Medium: 1)          | 6                 |
| P5          | 1 (Medium: 1)          | 15                | 1 (Low: 1)             | 15                |
| P6          | 0                      | N/A               | 1 (Medium: 1)          | 5                 |
| P7          | 0                      | N/A               | 1 (Medium: 1)          | 4                 |
| P8          | 0                      | N/A               | 3 (Medium: 1, Low: 2)  | 11                |
| P9          | 0                      | N/A               | 1 (Medium: 1)          | 5                 |
| P10         | 0                      | N/A               | 1 (Medium: 1)          | 7                 |
| P11         | 0                      | N/A               | 1 (Medium: 1)          | 4                 |
| P12         | 0                      | N/A               | 1 (Medium: 1)          | 3                 |

N/A indicates that no vulnerabilities were remediated within the allotted time; in these cases, the maximum task duration was assigned.

with security feedback in CI/CD workflows. Prior work shows conciseness if valued for development tools [12], and LLMs can provide concise and human-centric DAST alert summaries [54]. Research also shows the complexity of dynamic testing is a challenge for integration within CI/CD pipelines [41]. However, in both phases participants reported that summarized alerts were easier to understand and clearer than raw DAST outputs. By distilling complex scan results into focused, developer-oriented summaries, LLM-based summarization helped reduce the effort required to triage vulnerabilities and determine next steps. However, LLM hallucinations can be detrimental to software testing and security [35, 49]. Thus, our findings motivate future work to enhance LLM summarization to promote DAST integration in development workflows.

**Structured Reporting:** We found structured and interactive reporting mechanisms are valuable for vulnerability reporting. In our formative survey results, developers provided feedback to generate structured feedback in security alerts. This involved updating the generated report to be an interactive HTML document instead of a static text file and adding structure to the generated PR comment. In addition, structuring feedback around PR semantics (e.g., new, resolved, and existing findings) and security alert level helped developers reason about the security impact of their changes rather than treating vulnerabilities as static artifacts. Existing work shows security reports can invoke information overload for developers [8]. Prior work suggests improved organization of feedback, such as structured grouping of content, can reduce information overload [6]. This suggests future tools should provide structured feedback to mitigate information overload to support DAST tool integration.

## 7.2 Workflow Integration

**Context-Aware PR Integration.** Survey and user study participants consistently preferred *SafeAIMerge* due to its integration within PR-based workflows, which allows developers to engage with DAST findings in existing processes. Prior work suggests integration challenges prevents development tool adoption [11, 25]. Integration within PRs has shown to enhance tool integration in development workflows, such as dependency management [34] and

static analysis [47]. Further, *SafeAIMerge* provides context-aware suggestions by conducting dual DAST scans on the PR and main versions of the codebase. This provides capabilities to newly introduced issues, issues resolved in the PR, and persistent issues. Prior work shows context-aware analysis based on code changes enhances automatically generated software patches and testing [30, 63]. This shows context-aware PR integration can support human-centered designs for integrating DAST security systems in development workflows.

**Actionable Remediation:** Our findings highlight the importance of presenting security findings in a manner that supports vulnerability remediation rather than vulnerability diagnosis alone. *SafeAIMerge* prompts underlying LLMs to not only summarize DAST reports, but also provide actionable guidance to address reported vulnerabilities. Participants responded positively to the generated step-by-step guidance for resolving reported vulnerabilities, and a substantially larger proportion of developers were able to remediate alerts more quickly compared to the baseline DAST workflow. This suggests that actionable remediation reduces cognitive overhead associated with interpreting security reports while enhancing vulnerability fixes. Prior work shows developers fail to act on complex security reports [48]. For instance, automated security testing tools often incur alert fatigue for developers [51]. Yet, actionable guidance can help programmers mitigate vulnerabilities in professional development settings [31]. Actionability has also been shown to reduce alert fatigue [23], such as in cloud monitoring systems [60] and distributed systems monitoring [59]. These results suggest that DAST tools intended for CI/CD integration should prioritize actionable, context-aware remediation guidance in feedback to developers.

## 8 Threats to Validity

Every participant worked through both experimental conditions in a single sitting. While counterbalancing was employed to mitigate sequence and learning effects, it remains possible that familiarity—whether with the task context, the setup, or the types of vulnerabilities encountered—transferred between conditions and shaped how

participants performed. Additionally, subjective and self-reported NASA-TLX workload scores and workflow preferences can be susceptible to expectancy effects or social desirability bias. Additionally, having the researcher present may have influenced participants' behavior and approach. To mitigate these concerns, all participants received standardized instructions, and were assured anonymity.

External validity is limited by both study's experimental setting and scope. The study relied on two purpose-built TypeScript web-apps, each configured to expose an identical and controlled set of security issues. Although this guaranteed consistent comparison across workflows and enabled an just evaluation, industrial software typically involve bigger codebases, intricate system designs, and a wide variety of vulnerability classes. Furthermore, we focused exclusively on one DAST scanner (ZAP) within a single CI/CD setup built around GitHub Actions, which could limit how well the results transfer to alternative security tools, CI/CD platforms, organizational environments, or development practices.

Construct and conclusion validity are shaped by task success, workload and performance. Our user study placed its primary focus on brief (15-minute) remediation performance, self-assessed cognitive effort, and tool preference. The timeframe was enforced to ensure our study was completed in a timely manner. Although these metrics are commonly used in HCI research, they fall short of reflecting longer effects including sustained gains in organizational adoption, developer productivity, or gradual strengthening of an organization's security stance. Furthermore, all participants in the baseline condition reached the upper time limit, resulting in uniformly capped completion times that made inferential statistical analysis unsuitable for duration-based comparisons due to which the findings are presented in a descriptive manner.

## 9 Future Work

The results of this study suggest multiple avenues for continued research on incorporating LLM-enhanced security tools into developer-centered workflows. One immediate direction is to conduct larger-scale evaluations with broader participant pools spanning a wider range of development environments, job functions, and levels of expertise. Research of this kind will support assessment of whether the observed reductions in perceived workload, improvements in task performance, and workflow preferences extend beyond the controlled experimental setting. Future evaluations should also deploy *SafeAIMerge* on production-scale repositories with more complex codebases and system architectures. In contrast to the intentionally designed applications used in this study to maintain alert parity, real-world systems typically produce more voluminous and heterogeneous security findings and exhibit deeper interdependencies among services and settings. Investigating how PR-aware, summarized security reports performs under such conditions represents an important direction for future inquiry.

Longitudinal and in situ studies constitute a second key research direction. Examining developer interactions with *SafeAIMerge* over longer development periods would enable analysis of learning curves, sustained usability, and longer-term adoption patterns. Field-based deployments would also allow researchers to explore broader outcomes such as changes in how developers work and coordinate, and organizational security workflows. In addition, future research

could consider other experimental setups and evaluation criteria. For example, extending or adapting time constraints could facilitate more fine-grained comparisons of remediation efficiency, while incorporating objective security measures—such as vulnerability reintroduction rates, remediation accuracy, or downstream security impact—could complement subjective workload assessments. Integrating controlled studies with quantitative security outcome analyses would further strengthen evidence for the effectiveness of LLM-assisted security approaches.

Lastly, our introduced approach is limited to DAST or web application security. Exploring the application of LLM-based summarization and contextualized feedback to other security practices, including static analysis, dependency vulnerability scanning, and infrastructure-as-code security, may broaden the relevance of this approach. Collectively, these directions point toward the development of more human-centered, context-aware security tooling that is better aligned with contemporary Agile and CI/CD development processes.

## 10 Conclusion

This study presents *SafeAIMerge*, a tool that leverages LLMs within CI/CD pipelines to provide developers with context-rich security insights as part of the code reviews. By merging DAST outputs with PR-aware summarization and actionable fix suggestions, our tool seeks to bring security analysis closer to developers' routine workflows. Our evaluation followed a two-phase, mixed-methods design. In the first phase, a web-based survey revealed that participants unanimously reported *SafeAIMerge* as easy to use, naturally fitting within CI/CD, and beneficial for receiving security feedback directly PRs—giving it a high adoption likelihood. These outcomes guided enhancements to the tool's integration and reporting capabilities. During the second phase, a controlled within-subjects experiment compared *SafeAIMerge* against standard ZAP workflow. The findings revealed that summarized, PR-contextual security insights led to higher task completion rates, a significant decrease in perceived cognitive and emotional load and faster resolution of vulnerabilities. *SafeAIMerge* was unanimously preferred. In addition to the performance improvements, our results underscore the significance of designing security tools with developer experience in mind. Practical summaries, straightforward fix suggestions, and tight coupling with CI/CD pipelines and PR workflows allowed developers to progress more rapidly from discovering a vulnerability to fixing it, all while easing mental and emotional burden. Although evaluated under controlled settings, this research delivers both a tool and empirical proof that context-aware, LLM-powered security reporting can enhance how developers experience and perform the security issue remediation, motivating further research on human-centered, AI-assisted security integration in Agile and CI/CD-driven development.

## References

- [1] Oluwatosin Oluwatimileyin Abiona, Olutwatayo Jacob Oladapo, Oluwale Temidayo Modupe, Oyekunle Claudius Oyeniran, Adebunmi Okechukwu Adewusi, and Abiola Moshood Komolafe. 2024. The emergence and importance of DevSecOps: Integrating and reviewing security practices within the DevOps pipeline. *World Journal of Advanced Engineering Technology and Sciences* 11, 2 (2024), 127–133.

- [2] Vaibhav Agrawal and Kiarash Ahi. 2025. Llm-driven sast-genius: A hybrid static analysis framework for comprehensive and actionable security. *arXiv preprint arXiv:2509.15433* (2025).
- [3] M. A. Akbar, A. A. Khan, S. Mahmood, and S. Hyrynsalmi. 2025. Management of DevSecOps Process: An Empirical Investigation. *Software: Practice and Experience* 55 (2025), 1234–1255. Issue 7. doi:10.1002/spe.3419
- [4] SAIBS Arachchi and Indika Perera. 2018. Continuous integration and continuous delivery pipeline automation for agile software project management. In *2018 Moratuwa Engineering Research Conference (MERCon)*. IEEE, 156–161.
- [5] G. Atluri. 2025. The Invisible War on the Chain: Cyber Defense Strategies for Enterprise Blockchain Security. *European Journal of Computer Science and Information Technology* 13 (2025), 112–122. Issue 26. doi:10.37745/ejcsit.2013/vol13n26112122
- [6] Denise Atwood and Randy Uttley. 2011. Help! Strategies for preventing information overload. *Nursing management* 42, 7 (2011), 50–52.
- [7] Dejan Baca and Bengt Carlsson. 2011. Agile development with security engineering activities. In *Proceedings of the 2011 international conference on software and systems process*. 149–158.
- [8] Tridib Bandyopadhyay and Humayun Zafar. 2017. Influence of information overload on it security behavior: A theoretical framework. (2017).
- [9] Vijay Bhasker Reddy Bhimanapati and Shalu Jain. 2024. Security Testing for Mobile Applications Using AI and ML Algorithms. *Shalu, Security Testing for Mobile Applications Using AI and ML Algorithms (August 30, 2024)* (2024).
- [10] Larissa Braz and Alberto Bacchelli. 2022. Software security during modern code review: The developer's perspective. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 810–821.
- [11] Chris Brown and Chris Parnin. 2019. Sorry to bother you: Designing bots for effective recommendations. In *2019 IEEE/ACM 1st International Workshop on Bots in Software Engineering (BotSE)*. IEEE, 54–58.
- [12] Chris Brown and Chris Parnin. 2020. Comparing Different Developer Behavior Recommendation Styles. In *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops (Seoul, Republic of Korea) (IC-SEW'20)*. Association for Computing Machinery, New York, NY, USA, 78–85. doi:10.1145/3387940.3391481
- [13] Daniela Soares Cruzes, Michael Felderer, Tosin Daniel Oyetooyan, Matthias Gander, and Irđin Pekarić. 2017. How is security testing done in agile teams? A cross-case analysis of four software teams. In *International Conference on Agile Software Development*. Springer International Publishing Cham, 201–216.
- [14] Joost CF de Winter. 2014. Controversy in human factors constructs and the explosive use of the NASA-TLX: a measurement perspective. *Cognition, technology & work* 16, 3 (2014), 289–297.
- [15] Alexandre Decan, Tom Mens, Pooya Rostami Mazrae, and Mehdi Golzadeh. 2022. On the use of GitHub actions in software development repositories. In *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 235–245.
- [16] Lyubka Dencheva. 2022. *Comparative analysis of Static application security testing (SAST) and Dynamic application security testing (DAST) by using open-source web application penetration testing tools*. Ph.D. Dissertation. Dublin, National College of Ireland.
- [17] Sébastien Dupont, Guillaume Ginis, Mirko Malacario, Claudio Porretti, Nicolò Maunero, Christophe Ponsard, and Philippe Massonet. 2021. Incremental common criteria certification processes using DevSecOps practices. In *2021 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, 12–23.
- [18] Karen Mercedes Goertzel, Theodore Winograd, Holly L McKinley, Lyndon Oh, Michael Colon, Thomas McGibbon, Elaine Fedchak, and Robert Vienneau. 2007. Software security assurance: a state-of-art report (sar). *DTIC Document* (2007).
- [19] Mehdi Golzadeh, Alexandre Decan, and Tom Mens. 2022. On the rise and fall of CI services in GitHub. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 662–672.
- [20] Owen Graham and Kelvin Kloss. 2025. Evaluating the Impact of Reinforcement Learning on Autonomous CI/CD Workflow Optimization. (2025).
- [21] Muhammad Hammad, Irum Inayat, and Maryam Zahid. 2019. Risk management in agile software development: A survey. In *2019 international conference on frontiers of information technology (fit)*. IEEE, 162–1624.
- [22] Rashina Hoda, Norsareh Saleh, and John Grundy. 2018. The rise and evolution of agile software development. *IEEE software* 35, 5 (2018), 58–63.
- [23] Marilyn Hrvnak, Tiffany Pellathy, Lujie Chen, Artur Dubrawski, Anthony Wertz, Gilles Clermont, and Michael R Pinsky. 2018. A call to alarms: current state and future directions in the battle against alarm fatigue. *Journal of electrocardiology* 51, 6 (2018), S44–S48.
- [24] Hanlei Jin, Yang Zhang, Dan Meng, Jun Wang, and Jinghua Tan. 2024. A Comprehensive Survey on Process-Oriented Automatic Text Summarization with Exploration of LLM-Based Methods. *arXiv preprint arXiv:2403.02901* (2024).
- [25] Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge. 2013. Why don't software developers use static analysis tools to find bugs?. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 672–681.
- [26] Mete Keltek, Rong Hu, Mohammadreza Fani Sani, and Ziyue Li. 2024. Boosting Cybersecurity Vulnerability Scanning based on LLM-supported Static Application Security Testing. *arXiv preprint arXiv:2409.15735* (2024).
- [27] Timothy Kinsman, Mairieli Wessel, Marco A Gerosa, and Christoph Treude. 2021. How do software developers use GitHub Actions to automate their workflows?. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 420–431.
- [28] Amy J Ko, Thomas D LaToza, and Margaret M Burnett. 2015. A practical guide to controlled experiments of software engineering tools with human participants. *Empirical Software Engineering* 20, 1 (2015), 110–141.
- [29] Ahmed Lekssays, Hamza Mouhcine, Khang Tran, Ting Yu, and Issa Khalil. 2025. {LLMxCPG}:{Context-Aware} Vulnerability Detection Through Code Property {Graph-Guided} Large Language Models. In *34th USENIX Security Symposium (USENIX Security 25)*. 489–507.
- [30] Bo Lin, Shangwen Wang, Ming Wen, and Xiaoguang Mao. 2022. Context-aware code change embedding for better patch correctness assessment. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 3 (2022), 1–29.
- [31] Tamara Lopez, Helen Sharp, Thein Tun, Arosha Bandara, Mark Levine, and Bashar Nuseibeh. 2019. "Hopefully We Are Mostly Secure": Views on Secure Code in Professional Practice. In *2019 IEEE/ACM 12th International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE)*. IEEE, 61–68.
- [32] Maria Ilaria Lunesu, Roberto Tonelli, Lodovica Marchesi, and Michele Marchesi. 2021. Assessing the risk of software development in agile methodologies using simulation. *IEEE Access* 9 (2021), 134240–134258.
- [33] Grigori Melnik and Frank Maurer. 2005. A cross-program investigation of students' perceptions of agile methods. In *Proceedings of the 27th international Conference on Software Engineering*. 481–488.
- [34] Samim Mirhosseini and Chris Parnin. 2017. Can automated pull requests encourage software developers to upgrade out-of-date dependencies?. In *2017 32nd IEEE/ACM international conference on automated software engineering (ASE)*. IEEE, 84–94.
- [35] OWASP. 2025. LLM09: Overreliance. *GenAI Security Project* (2025). <https://genai.owasp.org/lmrisk2023-24/llm09-overreliance/>.
- [36] Yuanyuan Pan. 2019. Interactive application security testing. In *2019 International Conference on Smart Grid and Electrical Automation (ICSGEA)*. IEEE, 558–561.
- [37] Kai Petersen and Claes Wohlin. 2010. The effect of moving from a plan-driven to an incremental software development approach with agile practices: An industrial case study. *Empirical Software Engineering* 15 (2010), 654–693.
- [38] Xiao Pu, Mingqi Gao, and Xiaojun Wan. 2023. Summarization is (almost) dead. *arXiv preprint arXiv:2309.09558* (2023).
- [39] Roshan Namal Rajapakse, Mansoor Zahedi, and Muhammad Ali Babar. 2021. An Empirical Analysis of Practitioners' Perspectives on Security Tool Integration into DevOps. In *Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. 1–12.
- [40] K. Ramdass and S. Jain. 2025. The Role of DevSecOps in Continuous Security Integration in CI/CD Pipe. *Journal of Quantum Science and Technology* 2 (2025). Issue 1. doi:10.63345/jqst.v2i1.150
- [41] Thorsten Ragnau, Remco v Buijtenen, Frank Fransen, and Fatih Turkmen. 2020. Continuous security testing: A case study on integrating dynamic security testing tools in ci/cd pipelines. In *2020 IEEE 24th International Enterprise Distributed Object Computing Conference (EDOC)*. IEEE, 145–154.
- [42] Pilar Rodríguez, Mika Mäntylä, Markku Oivo, Lucy Ellen Lwakatare, Pertti Sepänen, and Pasi Kuvaja. 2019. Advances in using agile and lean processes for software development. In *Advances in Computers*. Vol. 113. Elsevier, 135–224.
- [43] Sinan Sakaoglu. 2023. KARTAL: Web Application Vulnerability Hunting Using Large Language Models: Novel method for detecting logical vulnerabilities in web applications with finetuned Large Language Models.
- [44] Mojtaba Shahin, Muhammad Ali Babar, and Liming Zhu. 2017. Continuous integration, delivery and deployment: a systematic review on approaches, tools, challenges and practices. *IEEE access* 5 (2017), 3909–3943.
- [45] Manish Sharma. 2021. Review of the benefits of DAST (dynamic application security testing) versus SAST. *International Journal of Management and Engineering Research* 1, 1 (2021), 05–08.
- [46] K Sinchana, C Sinchana, HL Gururaj, and BR Sunil Kumar. 2019. Performance evaluation and analysis of various network security tools. In *2019 International Conference on Communication and Electronics Systems (ICCES)*. IEEE, 644–650.
- [47] Devarshi Singh, Varun Ramachandra Sekar, Kathryn T Stolee, and Brittany Johnson. 2017. Evaluating how static analysis tools can reduce code review effort. In *2017 IEEE symposium on visual languages and human-centric computing (VL/HCC)*. IEEE, 101–105.
- [48] Justin Smith, Lisa Nguyen Quang Do, and Emerson Murphy-Hill. 2020. Why can't johnny fix vulnerabilities: A usability evaluation of static analysis tools for security. In *Sixteenth Symposium on Usable Privacy and Security (SOUPS 2020)*. 221–238.
- [49] Aditya K Sood, Sherali Zeadally, and EenKee Hong. 2025. The Paradigm of Hallucinations in AI-driven cybersecurity systems: Understanding taxonomy, classification outcomes, and mitigations. *Computers and Electrical Engineering* 124 (2025), 110307.

- [50] Benjamin Steenhoek, Kalpathy Sivaraman, Renata Saldivar Gonzalez, Yevhen Mohylevskyy, Roshanak Zilouchian Moghaddam, and Wei Le. 2024. Closing the Gap: A User Study on the Real-world Usefulness of AI-powered Vulnerability Detection & Repair in the IDE. *arXiv preprint arXiv:2412.14306* (2024).
- [51] Shahroz Tariq, Mohan Baruwal Chhetri, Surya Nepal, and Cecile Paris. 2025. Alert fatigue in security operations centres: Research challenges and opportunities. *Comput. Surveys* 57, 9 (2025), 1–38.
- [52] J. R. M. Thason and M. Prasad. 2025. Bridging the Gap: Integrating Security into DevOps Practices for Enhanced Software Delivery and Risk Mitigation. *International Journal of Research in Modern Engineering Emerging Technology* 13 (2025), 1–22. Issue 5. doi:10.63345/ijrmeet.org.v13.i5.130501
- [53] Arpit Thool and Chris Brown. 2024. Harnessing the Power of LLMs: LLM Summarization for Human-Centric DAST Reports. In *2024 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE.
- [54] Arpit Thool and Chris Brown. 2024. Harnessing the Power of LLMs: LLM Summarization for Human-Centric DAST Reports. In *2024 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE, 33–39.
- [55] Arpit Thool and Chris Brown. 2024. Securing Agile: Assessing the Impact of Security Activities on Agile Development. In *International Workshop on Software Security: Challenges, Opportunities, and Lessons Learned*.
- [56] Arpit Thool and Chris Brown. 2025. Integrating DAST in Kanban and CI/CD: A Real World Security Case Study. *arXiv preprint arXiv:2503.21947* (2025).
- [57] Arpit Thool and Dwayne Brown. 2025. Supplemental Material for SafeAIMerge: ZAP LLM CI/CD Tool. doi:10.6084/m9.figshare.30228256
- [58] Polona Tominc, Dijana Oreški, and Maja Rožman. 2023. Artificial intelligence and agility-based model for successful project implementation and company competitiveness. *Information* 14, 6 (2023), 337.
- [59] Krishna Vinnakota and Madhuri Kolla. 2025. Creating Effective Alerts for Monitoring Distributed Systems. *International Journal of Computer Trends and Technology* 73, 5 (2025), 172–178.
- [60] Fotios Voutsas, John Violos, and Aris Leivadeas. 2024. Mitigating alert fatigue in cloud monitoring systems: A machine learning perspective. *Computer Networks* 250 (2024), 110543.
- [61] Jonas Wagner. 2024. Analysis of security findings and reduction of false positives through large language models. (2024).
- [62] Charles Weir, Awais Rashid, and James Noble. 2020. Challenging software developers: dialectic as a foundation for security assurance techniques. *Journal of Cybersecurity* 6, 1 (2020), tyaa007.
- [63] Ming Wen, Junjie Chen, Rongxin Wu, Dan Hao, and Shing-Chi Cheung. 2018. Context-aware patch generation for better automated program repair. In *Proceedings of the 40th international conference on software engineering*. 1–11.
- [64] Mairieli Wessel, Joseph Vargovich, Marco A Gerosa, and Christoph Treude. 2022. GitHub Actions: The Impact on the Pull Request Process. *arXiv preprint arXiv:2206.14118* (2022).
- [65] Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo Sun, and Yue Zhang. 2024. A survey on large language model (llm) security and privacy: The good, the bad, and the ugly. *High-Confidence Computing* (2024), 100211.
- [66] Mohamed Zain, Raduan Che Rose, Iskandar Abdullah, and Maslin Masrom. 2005. The relationship between information technology acceptance and organizational agility in Malaysia. *Information & management* 42, 6 (2005), 829–839.
- [67] Xin Zhou, Duc-Manh Tran, Thanh Le-Cong, Ting Zhang, Ivana Claire Irsan, Joshua Sumarlin, Bach Le, and David Lo. 2024. Comparison of static application security testing tools and large language models for repo-level vulnerability detection. *arXiv preprint arXiv:2407.16235* (2024).

Received 2026-02-05; accepted 2026-03-19